INTRODUCTION TO AI PROGRAMMING | PROJECT 1

Bird Species Classification Using Convolutional Neural Networks

Final Report - Feedback Revised Version

CNN-based fine-grained image classification for 200 bird species

Team Number: Group 02

Team Members: Eunseo Seo, Minjun Park, Gun Kim, Chaemin Lee

Course: Introduction to AI Programming

Semester: 2026 Spring

Revision summary. This version adds a front report-structure table, explains reproducibility using fixed random seeds, adds code screenshots for reproducibility and evaluation outputs, explains why experimental conditions changed between the baseline, ResNet, and EfficientNet stages, expands the motivation for each experiment, adds freezing/fine-tuning discussion, expands detailed evaluation and model comparison, and adds repository/deliverable guidance.

Report Structure Based on Assignment Section 8.2

The assignment recommends a clear ten-part report structure. This revised report follows that structure directly, while splitting the Results section into the required training results, detailed evaluation, and model comparison subsections.

Table 1. Report structure mapped to the assignment recommendation.

Recommended section	Where it appears in this report	Main required content covered
1. Introduction	Section 1	Problem goal, practical output, and why bird species classification is difficult
2. Dataset Description	Section 2	Dataset split, class count, image variation, and fine-grained difficulty
3. Methodology	Section 3	Overall workflow, reproducibility, fair-comparison logic, and experiment motivation
4. Model Architectures	Section 4	Baseline CNN, ResNet-18/50, EfficientNetV2-S, freezing/fine-tuning choices
5. Experimental Setup	Section 5	Training settings, optimizer/scheduler choices, seed control, and output files
6. Results	Section 6	Training curves, accuracy results, detailed evaluation, class-wise discussion, model comparison
7. Error Analysis	Section 7	Failure modes, confusion matrix interpretation, and qualitative examples
8. Discussion	Section 8	What worked, what did not work, limitations, repository, and deliverables
9. Conclusion	Section 9	Main findings and final model summary
10. References	Section 10	Dataset, model, optimizer, scheduler, and assignment references

Abstract

This project develops a convolutional neural network system for fine-grained bird species classification. The task uses a 200-class public bird image dataset with 11,788 images. The split used in the project contains 5,994 training images, 2,897 validation images, and 2,897 test images. Since each species has only about thirty training images on average, and many bird species look similar, the task is much harder than ordinary object classification.

The project followed the full deep learning workflow: dataset preparation, custom baseline CNN design, improved CNN experiments, transfer learning, hyperparameter tuning, quantitative evaluation, qualitative error analysis, and a Streamlit inference demo. The custom baseline CNN reached 48.8% test accuracy but showed clear overfitting. ResNet experiments improved the result to 75.7% after learning-rate tuning. EfficientNetV2-S gave the strongest result, and the final selected configuration reached 88.13% test accuracy.

This revised version adds more explanation about why each experiment was done, why the settings were not always identical across models, how reproducibility was controlled, how the models differ in complexity, and how the final repository should be organized. It also keeps the presentation figures inside the main report body on clean white backgrounds.

1. Introduction

Image classification is one of the main tasks in computer vision, but fine-grained classification is more difficult than general object recognition. In this project, the goal is not just to classify a picture as a bird. The model must identify one species among 200 bird classes. Many classes share similar shapes and colors, so the model

needs to learn small details such as feather pattern, beak shape, head color, wing markings, and body proportions.

The practical output of the system is a predicted species, a confidence score, and the top-3 candidate classes. Top-3 output is important because several bird species can look very similar. If the model is uncertain, showing the next possible species gives more useful information than showing only one label.

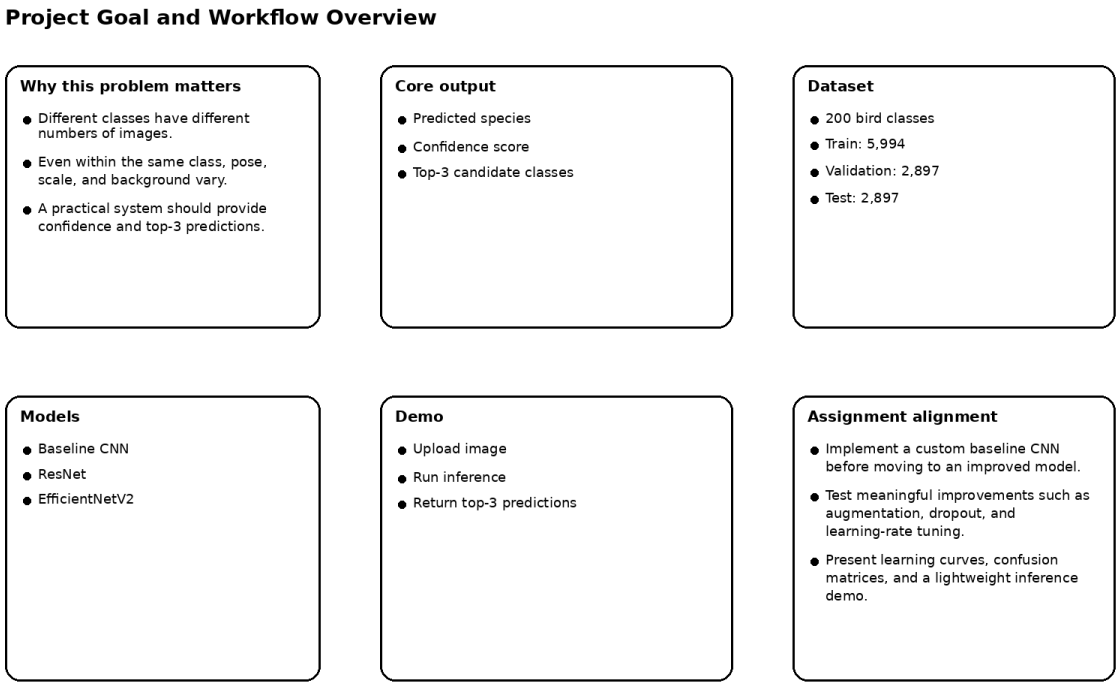

Figure 1. Project overview and workflow.

2. Dataset Description

The dataset is a public CUB-200-2011 style bird species dataset. It contains 200 classes and a total of 11,788 images. The project split uses 5,994 images for training and splits the original test portion into validation and final test sets. The validation and test sets each contain 2,897 images. The split was made with stratification so that class proportions stayed balanced as much as possible.

Table 2. Dataset split summary.

Split	Images	Approx. images per class	Role
Training	5,994	~30	Used to update model weights
Validation	2,897	~14-15	Used for tuning settings and checking overfitting
Test	2,897	~14-15	Used for final held-out accuracy

Figure 2. Dataset overview.

The dataset has both inter-class similarity and intra-class variation. Some species are visually close to each other, while images from the same species can have different pose, age, scale, background, and lighting. Some birds are also partly hidden by branches, water, or other objects. This is the main reason why a shallow CNN is likely to overfit and why transfer learning was important.

Figure 3. Examples of the main image-classification difficulties.

3. Methodology

3.1 Overall Workflow

The project was developed in stages instead of training only one model. We first built a custom CNN baseline to satisfy the baseline requirement and to understand the difficulty of the task. After the baseline overfit, we moved to pretrained ResNet models. After ResNet improved but still had a limited result, we moved to EfficientNetV2-S and tested several improvement strategies.

- Prepare the dataset, labels, bounding boxes, and train/validation/test split.
- Train a custom baseline CNN and inspect train/validation loss and accuracy.
- Improve the custom baseline by adding one more convolution layer and dropout.
- Fine-tune pretrained ResNet models and test dropout, augmentation, normalization, bounding boxes, and learning rates.
- Fine-tune EfficientNetV2-S and tune resolution, dropout, label smoothing, weight decay, optimizer, and scheduler settings.
- Evaluate the final model with test accuracy, learning curves, confusion matrix, classification report, and qualitative demo output.

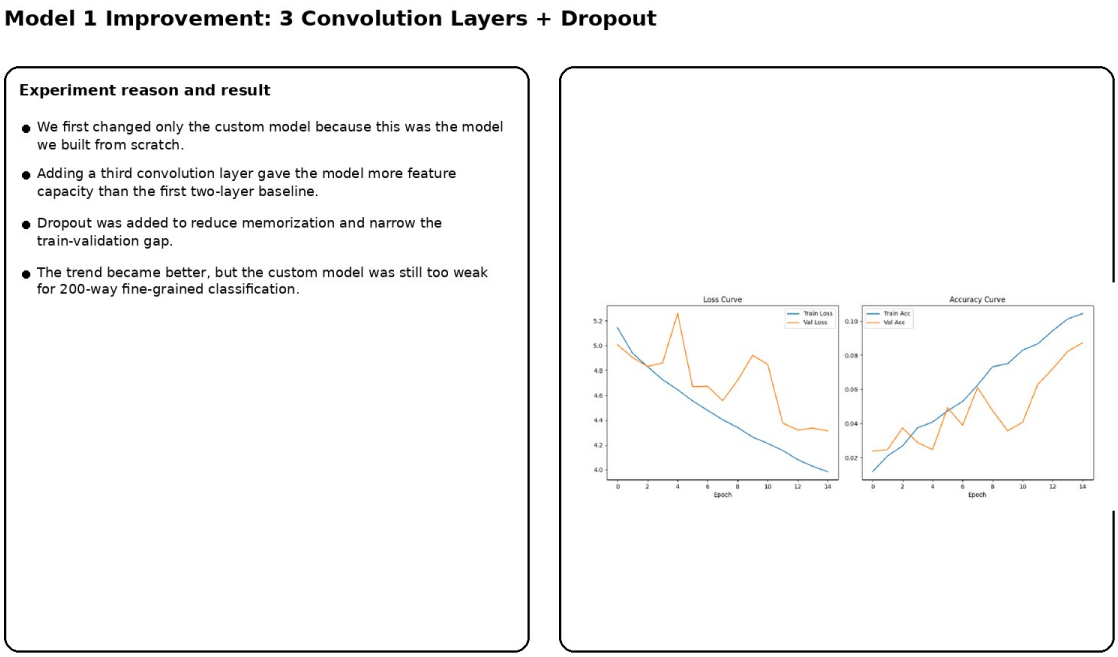

Figure 4. Motivation and result of the improved custom baseline experiment.

3.2 Reproducibility

The code fixed the random seed to improve reproducibility. The seed was set to 42 for Python random, NumPy, PyTorch, CUDA, and cuDNN. The validation/test split also used random_state=42. This does not make every hardware environment perfectly identical, but it makes the main random choices stable and makes the same code much easier to reproduce.

Code Figure: Reproducibility Settings

Random seed and deterministic GPU settings

```
# ================================================================================
# 0. □□ □□ □□
# ================================================================================
seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
if torch.cuda.is_available():
    torch.cuda.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

Stratified validation/test split with fixed random_state

```
    # Dataset Load & Split
    train_dataset = CUBDataset(root_dir=data_root, split='train', transform=train_transform)
    full_test_dataset = CUBDataset(root_dir=data_root, split='test', transform=eval_transform)

    test_labels = [item['label'] for item in full_test_dataset.data]
    test_indices = list(range(len(full_test_dataset)))

    val_idx, test_idx = train_test_split(
        test_indices,
        test_size=0.5,
        stratify=test_labels,
        random_state=42
    )
```

Figure 5. Code screenshot showing seed control and fixed validation/test split.

The final code also saves the trained model checkpoint, learning curves, confusion matrix, and classification report. These outputs make the result easier to inspect and reproduce. For the GitHub repository, the same seed and command-line arguments should be included in the README so that another user can rerun the experiment.

3.3 Fair Comparison and Why Conditions Changed

The experiments were not designed as a perfectly fixed benchmark where every model must use exactly the same image size and training setting. Instead, they were staged improvement experiments. To keep the comparison fair, all models used the same 200-class task, the same dataset source, the same label space, and the same final evaluation idea. However, the settings changed when the model capacity and experiment goal changed.

For example, the shallow baseline CNN was not trained at 384 x 384 resolution. The baseline model had limited depth and weak feature capacity, so using a very large input image would mostly increase computation and memory without guaranteeing better fine-grained feature learning. The goal of the baseline was to provide a simple from-scratch reference, not to compete with a pretrained high-resolution backbone.

ResNet was tested as the first transfer-learning model because it is a standard CNN backbone. The ResNet experiments focused on learning-rate stability, regularization, and preprocessing. EfficientNetV2-S was later tested because it can use parameters more efficiently and is better suited to detailed visual features. For EfficientNetV2-S, higher resolution was reasonable because the model was strong enough to use small details such as feathers, eye rings, and beak shape. Batch size was reduced from 64 to 32 when resolution increased because high-resolution images require more GPU memory.

Table 3. Fair-comparison logic and reason for different conditions.

Model stage	Main condition	Why this condition was reasonable
Baseline CNN	Lower-resolution/simple input, custom shallow CNN	Used to test the minimum from-scratch baseline; high resolution would raise cost more than useful capacity
Improved baseline	3 conv layers + dropout	Added feature capacity while trying to reduce memorization
ResNet	Pretrained backbone, learning-rate tuning	Used transfer learning but still kept training stable with smaller learning rates
EfficientNetV2-S	384 x 384 in later experiments, batch size 32	Used higher resolution because fine-grained details matter and the model can use them better
Final model	Dropout 0.3, label smoothing 0.1, weight decay, eta_min 0.00001	Combined settings that improved generalization without making the model underfit

3.4 Motivation for Each Experiment

The main direction of the experiments was to improve test accuracy while reducing overfitting. In practice, this meant watching the gap between training accuracy and validation accuracy, checking if validation loss became unstable, and testing whether the final test accuracy improved without making the model too complex or too slow.

Table 4. Motivation and intended purpose of each experiment.

Experiment	Why we tried it	Expected effect
3 conv layers + dropout	The first baseline was too shallow and overfit strongly	More feature capacity plus less memorization
ResNet pretrained backbone	Custom CNN features were too weak for 200 species	Better starting features from ImageNet
Dropout / augmentation / normalization / bounding box	Need to reduce background bias and train/validation gap	Better generalization and more bird-focused features
ResNet learning-rate tuning	Pretrained models can be damaged by too large a learning rate	Find a stable fine-tuning speed
EfficientNetV2-S	ResNet improved but still left a large performance gap	Stronger fine-grained feature extraction
Label smoothing	The model could become too confident on a small per-class dataset	Reduce overconfidence and improve calibration
Higher resolution	Species details can be small and local	Preserve feather, beak, eye, and wing details
RandAugment	The dataset has only about 30 training images per class	Increase visual diversity through augmentation
Weight decay	Large weights can increase memorization	Regularize parameters and reduce overfitting
Dropout sweep	Dropout 0.5 might be too strong or too weak	Find the best balance between overfitting and underfitting
Learning rate 0.0001	A larger LR might reach a better solution faster	Test speed and peak accuracy, while watching overfitting
SGD for 70 epochs	SGD can sometimes generalize well if trained longer	Test if longer SGD training beats Adam in our setting
Scheduler eta_min tuning	Late-stage learning rate may become too small	Keep updates active near the end of training

4. Model Architectures

4.1 Custom Baseline CNN

The baseline model was a custom CNN built from scratch. It used standard CNN components: convolution layers, ReLU activation, pooling, flattening, and fully connected layers for 200 output classes. No pretrained weights were used, so there was no freezing step. All weights were learned from the bird dataset.

The first baseline showed severe overfitting, so the manual improvement added a third convolution layer and dropout. The reason was simple: the model needed more feature capacity than the first two-convolution version, but it also needed regularization because there were only about thirty training images per class.

4.2 ResNet-18 and ResNet-50

ResNet was used as the first improved CNN family. ResNet-18 is lighter and faster, while ResNet-50 is deeper and more expressive. In both cases, the final classification layer had to be replaced with a 200-class output layer. The ResNet experiments were treated as fine-tuning experiments rather than a final frozen-feature setup. This was reasonable because bird species classification needs dataset-specific local details that may not be fully represented by frozen ImageNet features.

Freezing early layers can reduce training time and overfitting, but it also limits adaptation. Since the task is fine-grained, allowing the pretrained backbone to update was a better final choice. The risk of overfitting was handled through dropout, augmentation, normalization, bounding-box cropping, and learning-rate tuning.

4.3 EfficientNetV2-S

EfficientNetV2-S was the strongest final model. The final code loads pretrained EfficientNetV2-S weights and replaces the classifier with dropout plus a 200-class linear layer. The optimizer is created with model.parameters(), which means the whole model is fine-tuned, not only the final classifier head. This choice was important because the model needed to adapt pretrained general image features into bird-species-specific features.

EfficientNetV2-S was a good fit because it is designed to be accurate while still being relatively efficient. The higher-resolution experiments were especially useful for this architecture because the model could use small visual details better than the shallow baseline.

Table 5. Architecture, fine-tuning policy, and approximate model complexity.

Model	Layer / block style	Pretrained?	Frozen or fine-tuned?	Approx. parameters
Custom baseline CNN	2 conv blocks, then 3 conv + dropout trial	No	All layers trained from scratch	Not recorded exactly; small compared with pretrained models
ResNet-18	18-layer residual CNN	Yes	Fine-tuned after replacing final layer	~11.3M with 200-class head
ResNet-50	50-layer residual CNN with bottleneck blocks	Yes	Fine-tuned after replacing final layer	~23.9M with 200-class head
EfficientNetV2-S	Fused-MBConv / MBConv style efficient CNN blocks	Yes	Whole model fine-tuned; classifier replaced	~20.4M with 200-class head

5. Experimental Setup

The final EfficientNetV2-S code used bounding-box cropping, resizing, normalization, RandAugment, random crop, random horizontal flip, Adam or SGD optimizer depending on the argument, cross-entropy loss with label smoothing, and cosine annealing scheduling. The final selected setting used learning rate 0.00001, Adam, weight decay 0.0001, dropout 0.3, label smoothing 0.1, 384 x 384 input, batch size 32, T_max equal to the number of epochs, and eta_min = 0.00001.

The code records the time for each epoch and prints training loss, training accuracy, validation loss, validation accuracy, current learning rate, and epoch time. It also saves learning curves, a confusion matrix, a classification report, and model weights. These outputs connect the source code to the report evidence.

Code Figure: Timing and Detailed Evaluation Outputs

Epoch time logging

```
    for epoch in range(1, args.epochs + 1):
        # 각 에포크 시작 시간 기록
        epoch_start_time = time.time()
        # 각 에포크 학습 검증 완료 후 종료 시간 기록
        epoch_end_time = time.time()
        time_elapsed = epoch_end_time - epoch_start_time
        mins = int(time_elapsed // 60)
        secs = int(time_elapsed % 60)

        history_train_loss.append(train_loss)
        history_val_loss.append(val_loss)
        history_train_acc.append(train_acc)
        history_val_acc.append(val_acc)

        print(
            f"Epoch [{epoch}/{args.epochs}] Time: {mins}m {secs}s | LR: {current_lr:.6f} | "
            f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
            f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}"
        )
```

Final test prediction collection

```
    model.eval()
    test_correct, test_total = 0, 0
    all_preds, all_labels = [], []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            test_total += labels.size(0)
            test_correct += (predicted == labels).sum().item()

            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    test_acc = test_correct / test_total

    print("=" * 50)
    print("Final Test Accuracy:", round(test_acc, 4))
    print("=" * 50)
    torch.save(model.state_dict(), "./bird2-bird_aug_EffNetV2.pth")
    print("bird2-Model saved as bird_aug_EffNetV2.pth")
```

Confusion matrix and classification report outputs

```
    # Confusion Matrix
    # ============================================================================
    cm = confusion_matrix(all_labels, all_preds)

    plt.figure(figsize=(14, 12))
    sns.heatmap(cm, cmap="Blues", annot=False)
    plt.title("Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.savefig("bird2-cub_aug_EffNetV2_confusion_matrix.png")
    report = classification_report(all_labels, all_preds, digits=4)

    with open("bird2-cub_aug_EffNetV2_report.txt", "w") as f:
        f.write(report)
```

Figure 6. Code screenshot showing epoch timing and detailed evaluation outputs.

Table 6. Final selected training setup.

Setting	Final value or behavior	Reason
Input resolution	384 x 384	Keeps more small species details for fine-grained recognition
Batch size	32	Reduced for high-resolution memory cost
Optimizer	Adam	Worked better than SGD in the tested setting
Learning rate	0.00001	Stable fine-tuning; avoids damaging pretrained features
Dropout	0.3 in final selection	Less underfitting than 0.7 and less restrictive than 0.5
Weight decay	0.0001	Controls large weights and helps reduce memorization
Label smoothing	0.1	Reduces overconfident predictions
Scheduler	CosineAnnealingLR, eta_min 0.00001	Keeps late-stage updates active
Seed	42	Improves reproducibility

6. Results

6.1 Training Results

The baseline CNN reached 48.8% test accuracy, but the training accuracy became almost 100%. This means the model memorized the training images but did not generalize well to new examples. The confusion matrix and curves show why moving to a pretrained model was necessary.

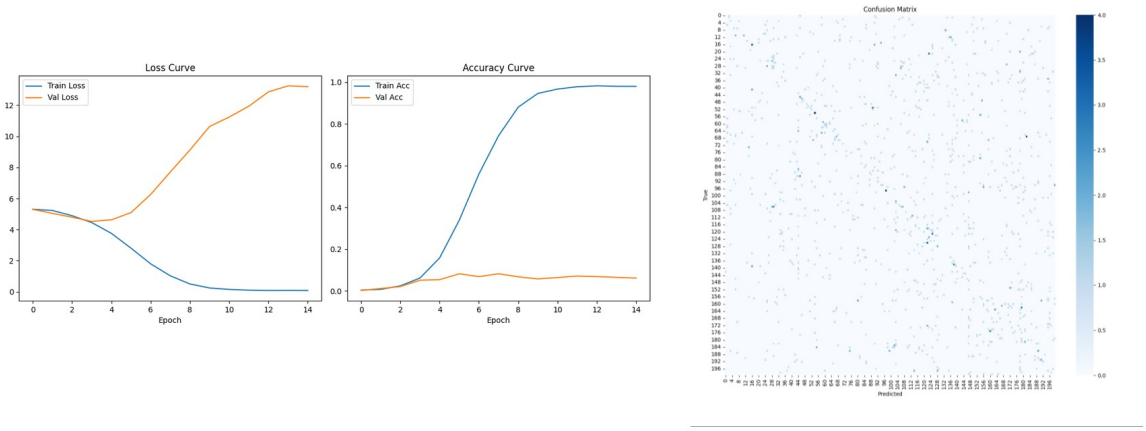

Figure 7. Baseline CNN learning curves and confusion matrix.

The ResNet experiments improved step by step. ResNet alone reached 48.94%, which was close to the custom baseline. Adding dropout improved it slightly to 50.30%. Adding dropout, augmentation, normalization, and bounding-box use improved the result to 59.53%. The learning-rate experiment was the largest ResNet improvement, and the best ResNet setting reached 75.7%.

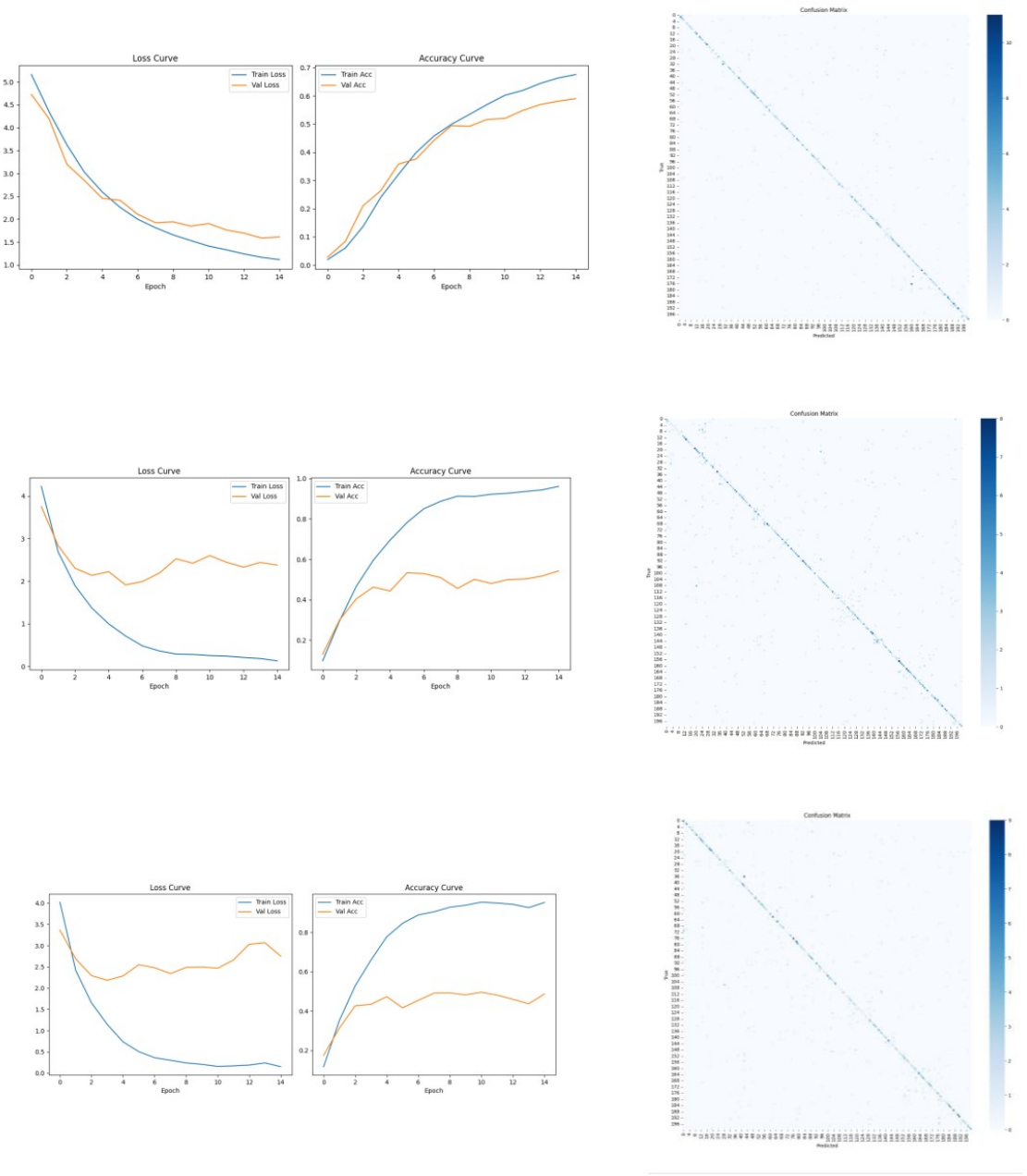

Figure 8. ResNet comparison panels.

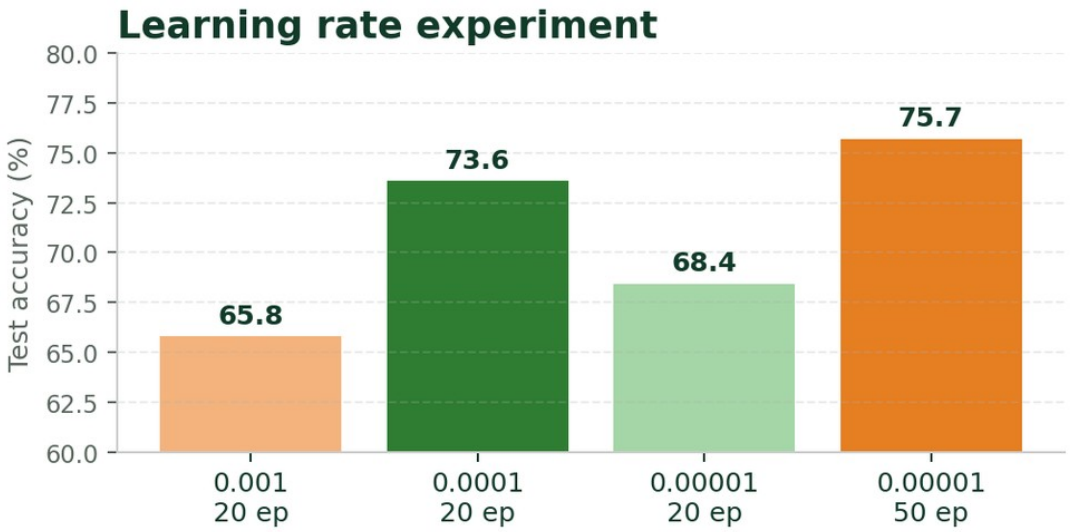

Figure 9. ResNet learning-rate experiment.

EfficientNetV2-S produced the largest jump. The initial EfficientNetV2-S result was 84.5%. After that, several experiments were tested to control overfitting and improve the final result.

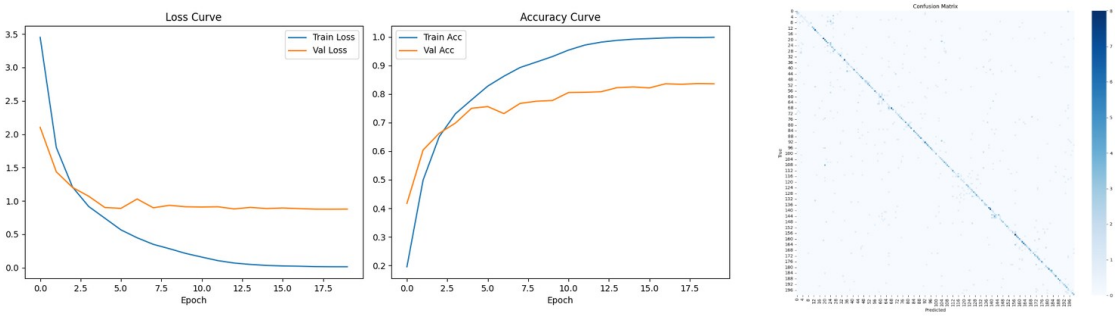

Figure 10. Initial EfficientNetV2-S learning curves and confusion matrix.

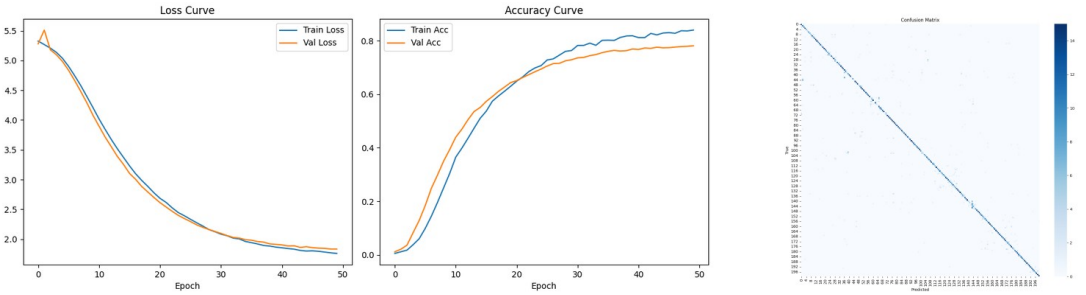

Figure 11. EfficientNetV2-S label smoothing experiment.

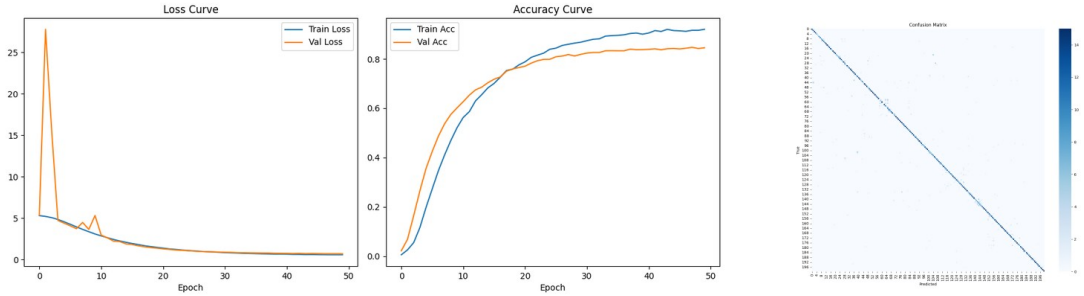

Figure 12. EfficientNetV2-S higher-resolution experiment.

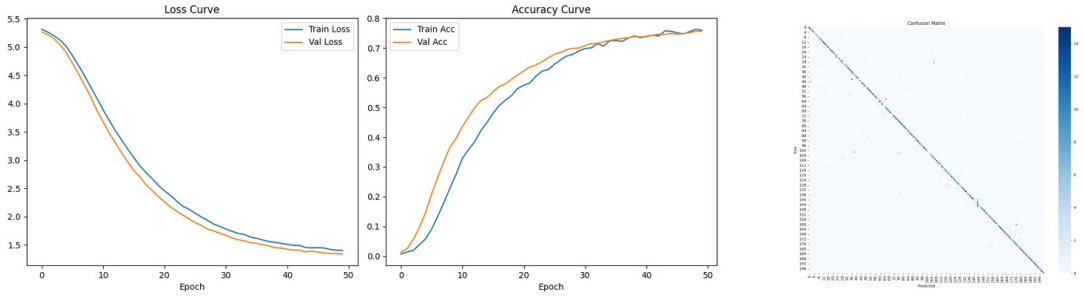

Figure 13. EfficientNetV2-S auto-augmentation experiment.

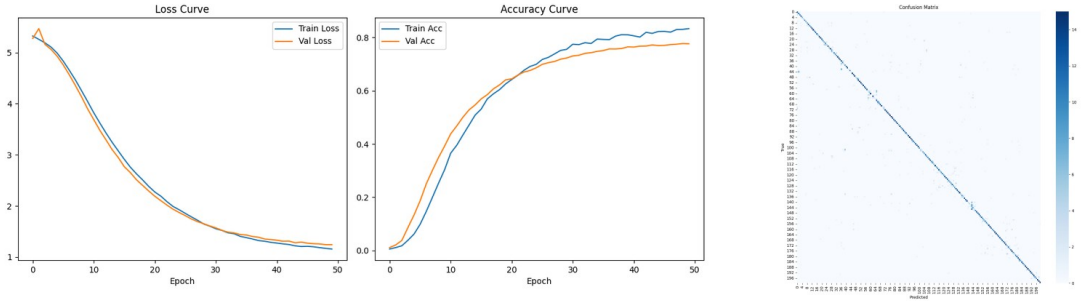

Figure 14. EfficientNetV2-S Adam with weight decay experiment.

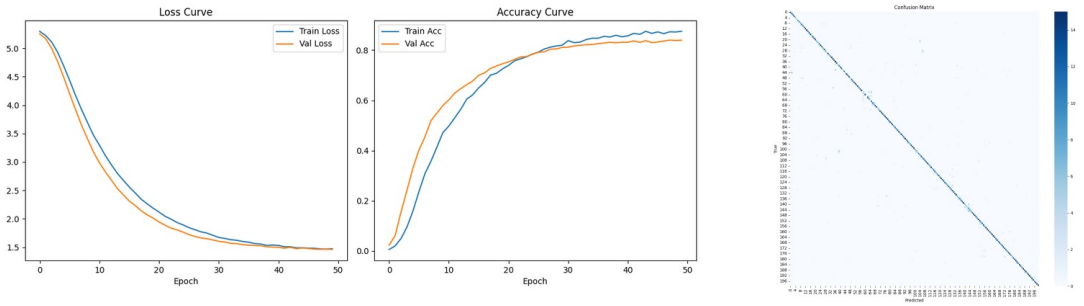

Figure 15. EfficientNetV2-S default high-resolution setting.

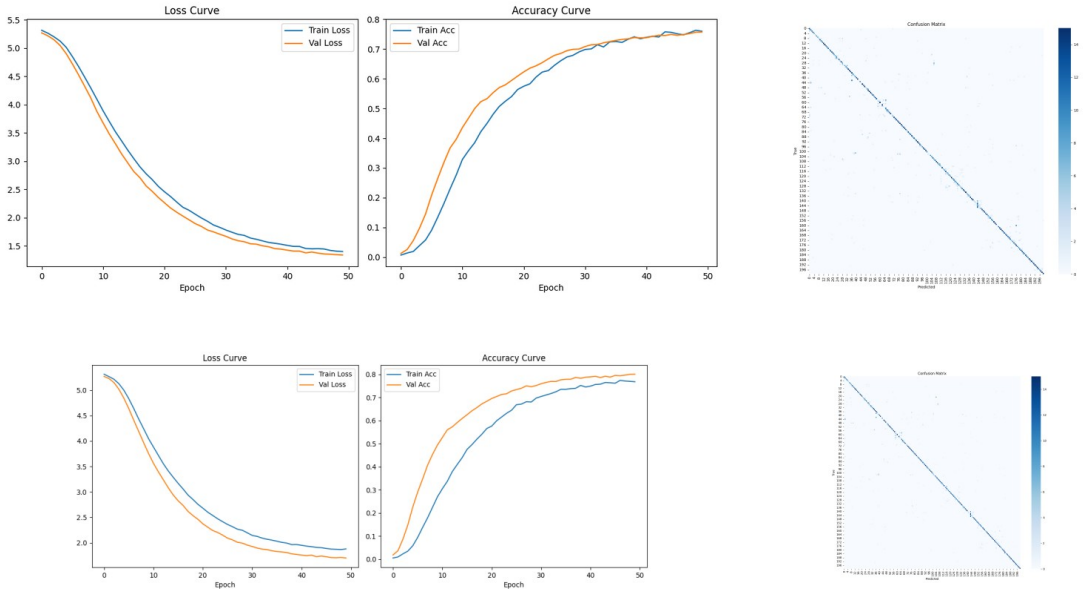

Figure 16. EfficientNetV2-S dropout experiments.

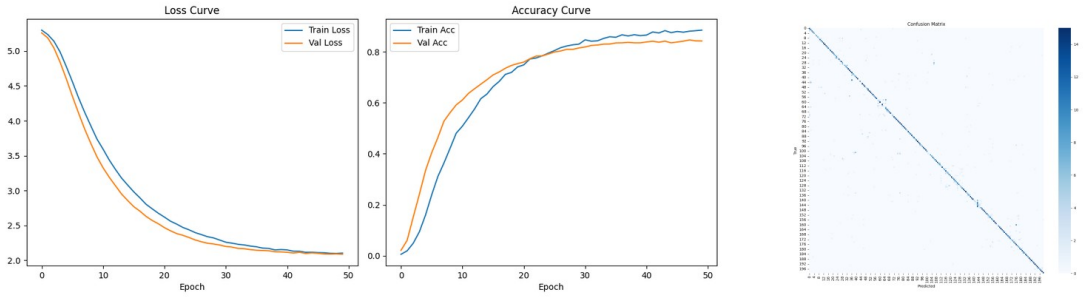

Figure 17. EfficientNetV2-S label smoothing 0.2 experiment.

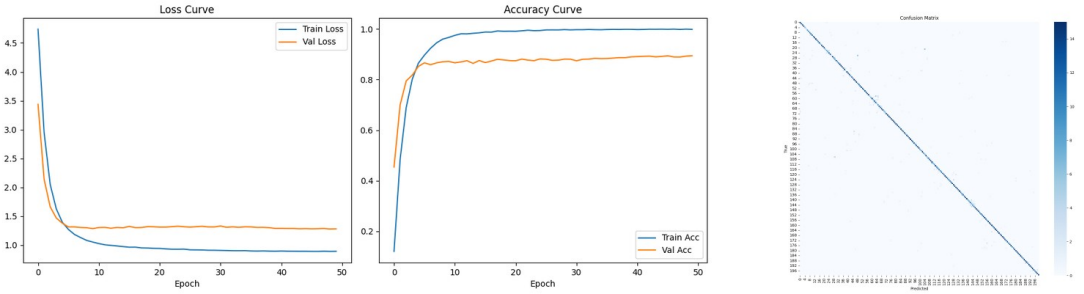

Figure 18. EfficientNetV2-S learning rate 0.0001 experiment.

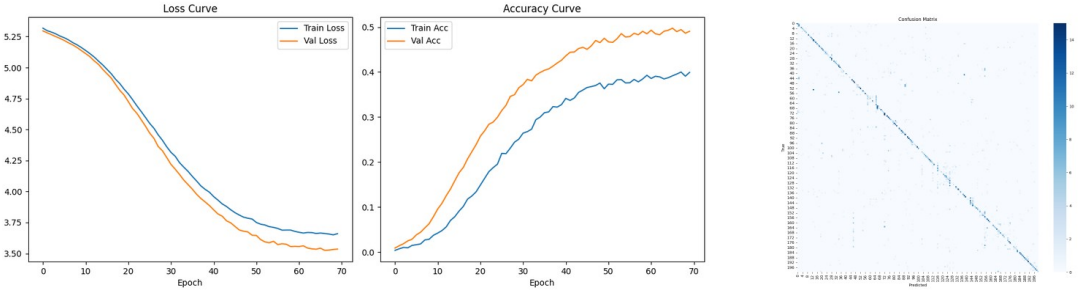

Figure 19. EfficientNetV2-S SGD optimizer experiment.

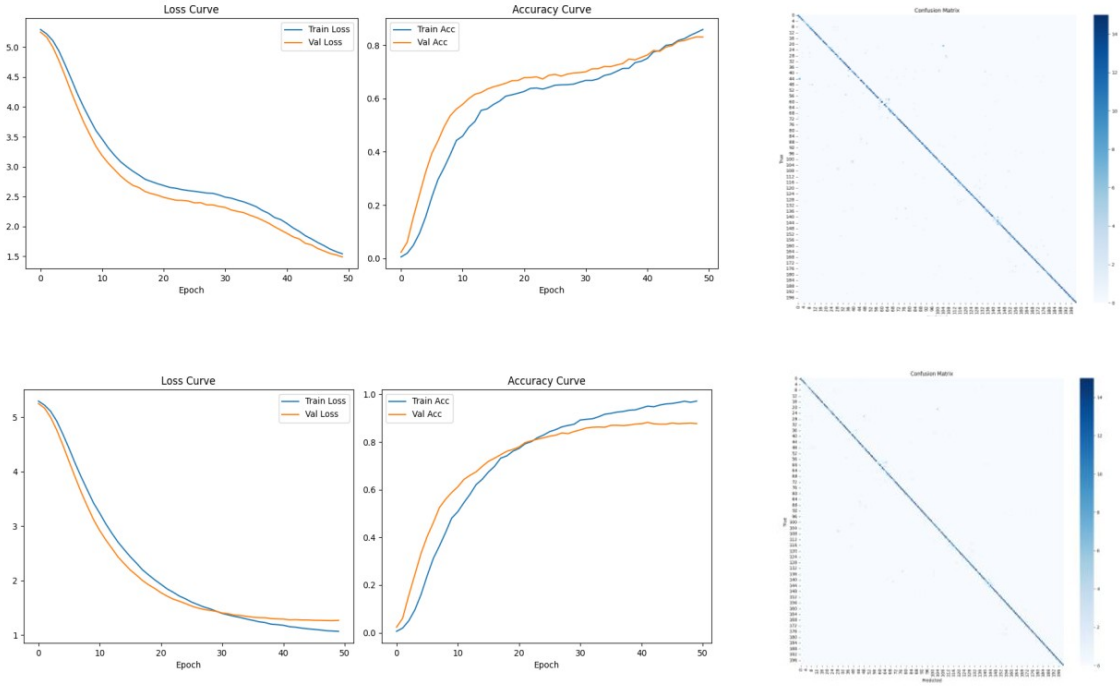

Figure 20. EfficientNetV2-S scheduler experiments.

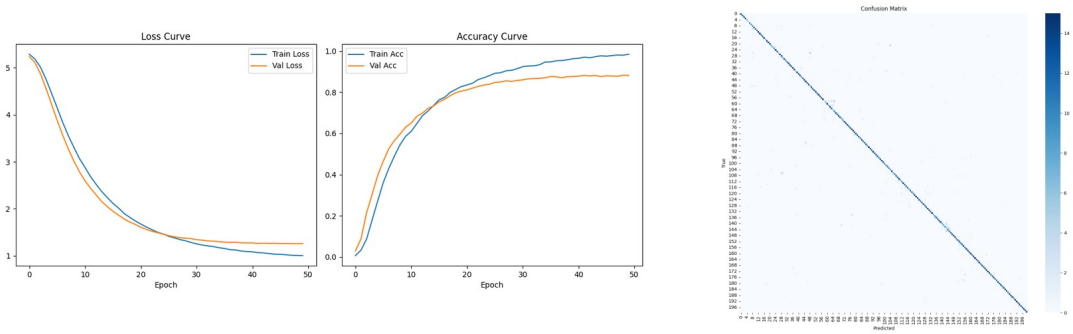

Figure 21. Final selected EfficientNetV2-S model result.

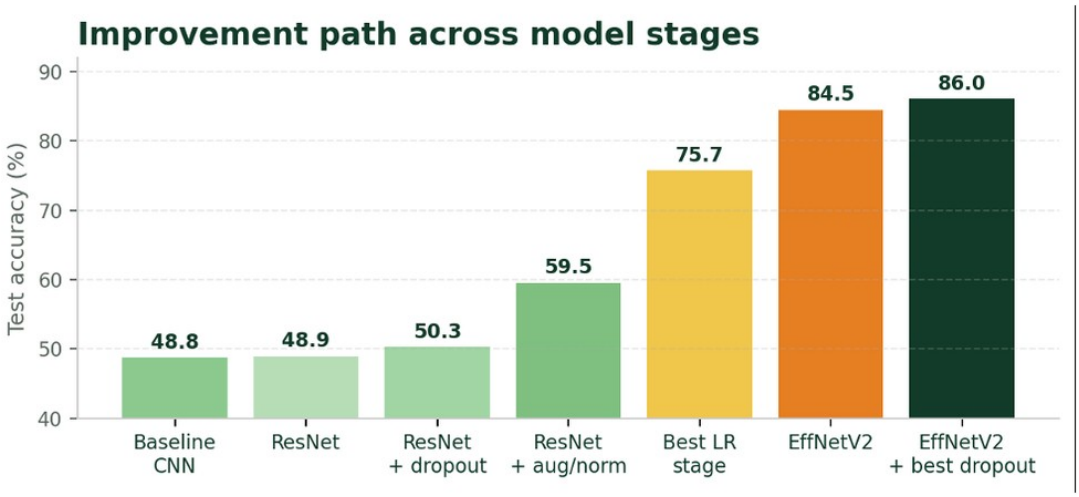

Figure 22. Overall improvement path across model stages.

Table 7. Main results across model stages.

Model / experiment	Main change	Test accuracy	Main interpretation
Baseline CNN	Custom CNN from scratch	48.8%	Learned training data but overfit strongly
ResNet only	Pretrained ResNet backbone	48.94%	Pretraining alone was not enough
ResNet + dropout	Added dropout	50.30%	Small regularization gain
ResNet + dropout/augmentation/normalization/bounding box	Stronger preprocessing and training strategy	59.53%	Meaningful improvement
Best ResNet LR stage	LR = 0.00001 for 50 epochs	75.7%	Small LR needed longer training
Initial EfficientNetV2-S	Architecture change	84.5%	Major representation improvement
Dropout 0.3	Reduced dropout from 0.5	85.5%	Better balance than heavier dropout
Learning rate 0.0001	10x larger LR	88.3%	Highest isolated value but stronger overfitting
Scheduler eta_min 0.00001	Higher minimum LR	87.5%	Late-stage updates helped
Final selected model	Dropout 0.3 + eta_min 0.00001 + 384 x 384	88.13%	Best accuracy-generalization trade-off

6.2 Detailed Evaluation

The final code generates a confusion matrix and a classification report. The confusion matrix is useful for seeing whether predictions concentrate on the diagonal. A stronger diagonal means that most images are classified into the correct species. The classification report gives precision, recall, and F1-score for each class, so it is the main tool for class-wise performance analysis.

From the observed confusion matrices, performance improved as the model moved from baseline to EfficientNetV2-S. The baseline matrix was more scattered, which matches the overfitting problem. The final EfficientNetV2-S matrix showed a stronger diagonal, but some off-diagonal errors remained. These errors are expected because many classes are visually similar and each class has a small number of examples.

Table 8. Detailed evaluation outputs and their purpose.

Detailed evaluation output	How it is produced	Why it matters
Learning curves	Saved as bird2-cub_aug_EffNetV2_learning_curve.png	Shows overfitting, underfitting, and training stability
Confusion matrix	Saved as bird2-cub_aug_EffNetV2_confusion_matrix.png	Shows class-level error patterns
Classification report	Saved as bird2-cub_aug_EffNetV2_report.txt	Gives precision, recall, and F1-score for every class
Correct examples	Generated by supplementary evaluation script from all_preds/all_labels	Shows what the model handles well
Failure examples	Generated by supplementary evaluation script from all_preds/all_labels	Shows common practical failure cases
Top confusion pairs	Computed from confusion_matrix(all_labels, all_preds)	Identifies the class pairs that are mixed up most often

For class-wise performance, the strongest classes are expected to be the ones with distinctive shape or color cues and clear bird visibility. The weakest classes are expected to be visually similar species, especially when the bird is small, occluded, juvenile, or placed in a complex background. The updated final code saves correct_cases.png, failure_cases.png, top5_best_recall.csv, top5_worst_recall.csv, and top10_confusion_pairs.csv. These outputs make the detailed evaluation more concrete because they connect the confusion matrix and classification report to actual image examples.

Table 9. Class-wise interpretation used for detailed evaluation.

Class-wise pattern	Typical reason	Expected model behavior
High recall classes	Distinctive shape, color, or markings; clear centered bird	Correct top-1 prediction more often
Medium recall classes	Recognizable bird but background or pose changes are present	Often correct, but top-3 may include similar species
Low recall classes	Small visual differences between related species	More off-diagonal confusion in the matrix
Hard failure cases	Occlusion, unusual pose, juvenile appearance, or distracting objects	Wrong top-1 prediction, but sometimes semantically related top-3 candidates

6.2.1 Top-5 Correct and Failure Case Analysis

After adding the qualitative case-output code, we inspected five correct predictions and five failure cases from the final model. These examples are useful because accuracy alone does not show what the model is actually learning. The figures currently display class IDs rather than species names, so the discussion below uses true and predicted IDs. If the final repository adds the CUB class-name mapping, the same cases can be printed with species names as well.

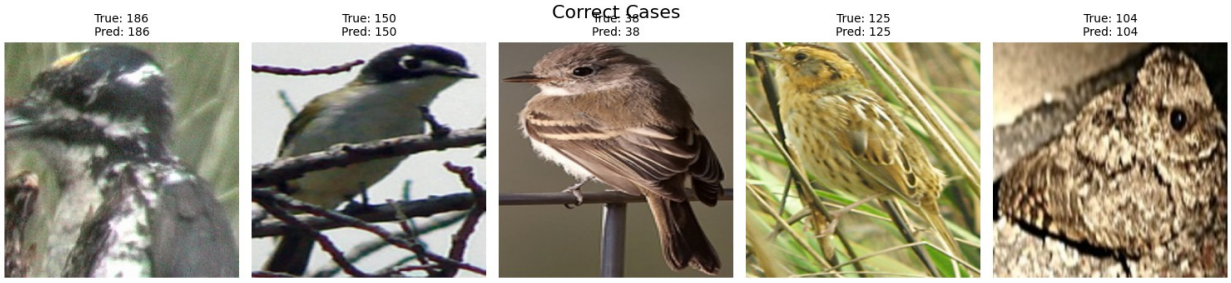

Figure 23. Top-5 correct prediction examples generated by the updated final evaluation code.

Table 10. Top-5 correct case interpretation.

Case	True/Pred ID	Visual reason the model handled it well	Meaning for evaluation
1	186 / 186	The image is low-quality and somewhat blurred, but the bird fills much of the crop and has clear dark-light body markings.	Shows some robustness to blur and imperfect image quality.
2	150 / 150	Branches partially cover the bird, but the head and body color pattern remain visible.	Suggests the model can still use bird-part cues under partial occlusion.
3	38 / 38	The bird is centered with a clean side pose and relatively simple background.	This is a typical easy case where shape and feather pattern are visible.
4	125 / 125	The bird appears inside grass, but the crop keeps the bird visible and the texture pattern is still strong.	Shows that bounding-box cropping and high resolution help with complex backgrounds.
5	104 / 104	The bird is close to the camera and the local texture is very clear.	Large object size and detailed texture make the prediction easier.

The correct examples are not all perfectly clean images. Some include blur, branches, grass, or close-up texture. This is encouraging because it suggests that the final model did not only memorize ideal textbook-like bird images. Instead, it learned useful local visual cues such as head shape, feather texture, body contour, and color contrast.

Figure 24. Top-5 failure examples generated by the updated final evaluation code.

Table 11. Top-5 failure case interpretation.

Case	True/Pred ID	Likely reason for the wrong prediction	Meaning for evaluation
1	88 / 45	The bird is wet or in an unusual pose near water, so the silhouette and texture are less typical.	Unusual pose and background can weaken class-specific cues.
2	134 / 77	The image is clear, but the bird has a common side-view shape and color pattern shared by similar species.	Fine-grained confusion can happen even when the photo quality is good.
3	129 / 122	The bird is small and streaked, with strong background colors and similar feather markings.	Small local markings are easy to confuse between related classes.
4	38 / 42	This has the same true class ID as one correct example, but lighting, pose, and color appearance are different.	Shows intra-class variation: the model can get one class right in one condition and wrong in another.
5	155 / 152	The bird is a side-view small passerine with similar wing and face markings to the predicted class.	The error is likely between visually close classes, not random categories.

The failure cases show that the remaining errors are mostly fine-grained and visually reasonable. The model tends to fail when the bird is small, has an unusual pose, is affected by background clutter, or belongs to a group with very similar local markings. This matches the confusion-matrix interpretation: the final model is much stronger than the baseline, but it still struggles when the correct species depends on small features that are hidden or visually close to another class.

The presentation demo also shows why top-3 output is useful. When the input bird belongs to a visually related group, the model may place similar species near each other in probability. This is not perfect classification, but it is useful information for a user because the alternatives are often reasonable candidates.

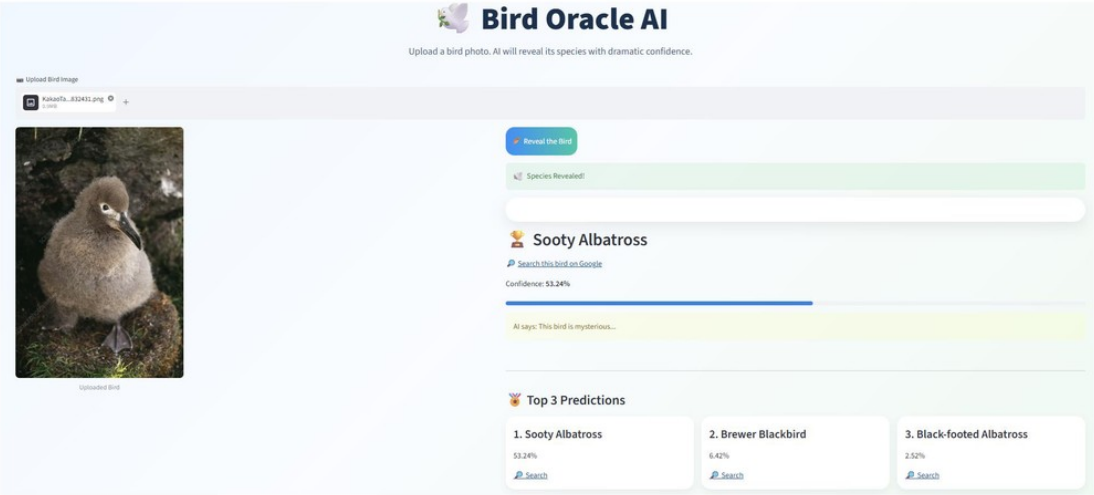

Figure 25. Streamlit inference demo with top-3 predictions.

6.3 Model Comparison

The final comparison should consider more than accuracy. It should also consider overfitting behavior, training time, model size, and whether the model is practical to train. The shallow baseline was the fastest and simplest, but it overfit. ResNet was stronger after tuning, but EfficientNetV2-S gave the best result for this fine-grained task.

Table 12. Training-time comparison. Early-stage exact logs were not kept, but final code now records epoch time.

Model / setting	Approx. per-epoch time	Approx. total time	Comment
Baseline CNN	Not logged exactly; fastest stage	Not logged exactly	Shallow model and lower input cost; timing should be logged in future reruns
ResNet stages	Not logged consistently	Not logged consistently	Moderate cost; main focus was accuracy and LR tuning
EfficientNetV2-S at 224 x 224	~35 sec / epoch	~29 min for 50 epochs	Fast enough for repeated trials
EfficientNetV2-S at 384 x 384	A little over 1 min / epoch	~50-60 min for 50 epochs	Higher resolution improved detail but increased cost
EfficientNetV2-S with SGD	A little over 1 min / epoch	~70-80 min for 70 epochs	Longer run, but accuracy was poor in our setting
Final EfficientNetV2-S	High-resolution cost range	~50-60 min for 50 epochs	Chosen for accuracy-generalization trade-off

Table 13. Model comparison by strength and weakness.

Model	Strength	Weakness	Best use in this project
Baseline CNN	Simple, fast, satisfies from-scratch requirement	Severe overfitting and weak representation	Reference point
ResNet-18	Lighter residual model	Less expressive than deeper models	Efficient transfer-learning trial
ResNet-50	Deeper residual features	More parameters and training cost	Stronger ResNet comparison
EfficientNetV2-S	Strong fine-grained performance with efficient parameter use	Higher-resolution runs are slower	Final selected model

7. Error Analysis

The main failure modes are caused by the nature of fine-grained bird classification. Many mistakes are not random. They happen when two species share similar colors and body shapes, or when the image hides the details needed for the correct label. The model is strongest when the bird is clear, centered, and visually distinctive. It is weakest when the bird is small, partly hidden, in a strange pose, or surrounded by distracting background objects.

Table 14. Main error modes and possible responses.

Failure mode	Why it is difficult	Model response or future fix
Similar species appearance	Bird species may differ only in small markings	Use higher resolution, attention, or more class-wise examples
Complex background	Branches, water, and shadows can dominate features	Use bounding boxes/cropping and better augmentation
Unusual pose or scale	Important features may not be visible	Use more diverse training examples
Age variation	Juveniles can look different from adults	Collect or augment age-varied examples
Occlusion / other objects	The bird is partly hidden or mixed with distractors	Use localization or attention methods
Small per-class dataset	Only about 30 train images per species increases memorization risk	Use transfer learning, dropout, weight decay, label smoothing, and class-balanced sampling

8. Discussion

8.1 What Worked

The most important improvement was moving from the custom CNN to pretrained models. The baseline showed that a shallow from-scratch model could learn training examples but could not generalize well. ResNet improved after learning-rate tuning, and EfficientNetV2-S improved the result much more because it extracted stronger visual features.

Higher resolution also made sense for fine-grained recognition. Small features such as feathers, eye rings, and beak shape can disappear at low resolution. However, resolution is not free. It increased the training time and forced a smaller batch size. Dropout 0.3 and eta_min 0.00001 were also important because they improved generalization without making the model underfit.

8.2 What Did Not Work as Well

Some experiments were reasonable but did not improve the final result. RandAugment did not outperform the strongest setting, possibly because aggressive transformations can damage the small details needed for species recognition. Dropout 0.7 was too strong and caused underfitting. SGD was also weak in this setting even though the number of epochs was increased to 70. This suggests that SGD may have needed a different learning rate, warmup, or momentum schedule to work well here.

8.3 Repository and Submission Package

The assignment also requires a GitHub repository with code and a clear README. The final repository should include the training code, evaluation code, inference demo code, saved figures, final report, presentation slides, and README. For the source-code item, the updated final script should be uploaded as final_train_eval_with_cases.py because it includes the seed setting, training loop, confusion matrix, classification report, class-wise output files, and top-5 correct/failure case visualization. The repository URL should be inserted before submission.

Table 15. Repository checklist based on assignment Section 8.4.

Repository item	Required content
GitHub link	Insert final URL here: https://github.com/<team>/<repository>
README.md	Project goal, dataset information, environment, dependencies, training, evaluation, and inference demo instructions
Training code	Dataset loading, preprocessing, model definition, training loop, seed setting, scheduler, checkpoint saving
Evaluation code	Learning curves, confusion matrix, classification report, top class-wise tables, correct/failure examples
Inference demo	Streamlit or similar interface for uploading an image and returning top-3 predictions
Outputs	Saved learning curves, confusion matrices, classification report, and final model checkpoint if allowed

Table 16. Final submission package checklist.

Final deliverable	What to submit
Source code	Training, evaluation, preprocessing, and demo code
Final report	PDF version of this report
Presentation	Project presentation slides
Repository	GitHub link with README and reproducible instructions

9. Conclusion

This project successfully built and evaluated a CNN-based bird species classifier for a 200-class fine-grained dataset. The custom baseline CNN satisfied the from-scratch model requirement but showed severe overfitting. ResNet improved the result after better training strategy and learning-rate tuning. EfficientNetV2-S was the strongest model, and the final selected setting achieved 88.13% test accuracy.

The final result improved by 39.33 percentage points compared with the baseline CNN. The main reasons for improvement were transfer learning, stronger architecture, higher resolution, moderate dropout, weight decay, label smoothing, and scheduler tuning. The project also includes a lightweight Streamlit inference demo, and the revised report now explains reproducibility, fairness, model complexity, experiment motivation, and repository preparation more clearly.

10. References

[1] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200-2011 Dataset," California Institute of Technology, Technical Report CNS-TR-2011-001, 2011.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.

[3] M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," Proceedings of the International Conference on Machine Learning, 2021.

[4] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical Automated Data Augmentation with a Reduced Search Space," CVPR Workshops, 2020.

[5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," International Conference on Learning Representations, 2015.

[6] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," International Conference on Learning Representations, 2017.

[7] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," International Conference on Learning Representations, 2019.

[8] R. Muller, S. Kornblith, and G. Hinton, "When Does Label Smoothing Help?" Advances in Neural Information Processing Systems, 2019.

[9] Introduction to AI Programming, "Project Assignment: CNN-Based Image Classification," 2026 Spring Semester.

[10] Group 02, "Bird Species Classification Using CNN," Project 1 presentation slides, 2026 Spring Semester.